

```
#####
# Understanding Artificial Intelligence through Algorithmic Information Theory #
# MOOC "AIAI" on EdX - see: https://aiai.telecom-paris.fr/ #
# Institut Mines Telecom 2021 - J.-L. Dessalles with Pierre-Alexandre Murena #
#####

"""
Computes compact binary codes for integers """

import sys
import os
from math import log, ceil
OUTPUTFILE = 'CompactCodes.csv'
__Correction = 1

def usage():
    return """
Usage: %s <value>
--> prints the shortest available code for <value>
Usage: %s
--> prints the shortest available codes for some range of values
Usage: %s <min> <max>
--> stores codes for values ranging from <min> to <max> into %s
=====
""" % ((os.path.basename(sys.argv[0]),) * 3 + (OUTPUTFILE,))

#####
# Short term memory (used in conditional compl)#
#####
def Memorize(CodedValue):
    global Mem
    Mem = CodedValue
    return Mem

def Recall():
    global Mem
    try:
        return Mem
    except NameError:
        return None

#####
# coding and decoding functions #
#####
def Binary(N):
    """ returns the binary code of N as a string """
    return bin(N)[2:]
    # bin(17) == '0b10001', so bin(17)[2:] == '10001'

def BinaryDecoding(SN):
    return int(SN, 2)

#-----#
# Compact coding #
# Same as binary coding, but without leading 1 #
#-----#

def CompactCoding0(N):
    """ Compact coding, as defined in https://aiai.telecom-paris.fr/videos/IntegerComplexity_film.webm """
    return Binary(N+2)[1:]

def CompactDecoding0(SN):
    return BinaryDecoding('1' + SN.strip()) - 2

def CompactCoding(N):
    """ Same as CompactCoding0, but with heading 1 """
    return '1' + CompactCoding0(N)

def CompactDecoding(SN):
    try:
        one, SN1 = SN.split(' ')
        one != '1' and int('') # trick to trigger ValueError if the code does not start with '1'
    except ValueError:
        raise Exception('CompactDecoding error with %s' % SN)
    return CompactDecoding0(SN1)

#-----#
# Special coding of round numbers #
#-----#

def RoundNumberCoding(N):
    """ Round numbers such as 30000 are coded as "0 <compact code of 3> <compact code of 4>" """
    SN = str(N)
    SN1 = SN.strip('0') # 123000 --> 123
    SN2 = SN[len(SN1):] # 123000 --> 000
    if SN1 and SN2: # really a round number
        # coding round number by: "0 L D"
        # where L is the compact code for the leading digits
        # and D is the compact code for the number of zeros
        # Note the use of white space separators, made possible by the compact code
        # (no syntactic ambiguity: after 0, one expects 2 arguments)
        return '0 %s %s' % (CompactCoding0(int(SN1)), CompactCoding0(len(SN2)))
    else:
        # standard compact coding
        return CompactCoding(N)

def RoundNumberDecoding(SN):
    try:
        zero, SN1, SN2 = SN.split(' ')
        zero != '0' and int('') # trick to trigger ValueError if the code does not start with '0'
        return CompactDecoding0(SN1) * 10 ** CompactDecoding0(SN2)
    except ValueError:
        # not a round number coding
        return CompactDecoding(SN)

def RoundNumberTest(N):
    NC = RoundNumberCoding(N)
    NCD = RoundNumberDecoding(NC)
    print('%d --> [%s] --> %d -- %s' % (N, NC, NCD, ('ERROR', 'ok')[1 * (N == NCD)]))

#-----#
# Coding numbers by their distance #
# to round numbers #
#-----#

def DifferenceCoding0(N, NRef):
    """ Difference is coded as "(00|01) <round coding of reference> <compact code of difference>" """
    if N > NRef:
        return '00 %s %s' % (RoundNumberCoding(NRef)[2:], CompactCoding0(N - NRef))
    if N == NRef:
        return RoundNumberCoding(N)
    if N < NRef:
        return '01 %s %s' % (RoundNumberCoding(NRef)[2:], CompactCoding0(NRef - N))

def DifferenceCoding(N, NRef):
    """ Same as DifferenceCoding, but with absolute coding of arguments """
    if N > NRef:
        return '000 %s %s' % (Coding(NRef), Coding(N - NRef))
    if N == NRef:
        return Coding(N)
    if N < NRef:
        return '001 %s %s' % (Coding(NRef), Coding(NRef - N))

def DifferenceDecoding(SN):
    Chunks = SN.split(' ')
    if Chunks[0] not in ['00', '01', '000', '001']:
        raise Exception('DifferenceDecoding error with %s' % SN)
    try:
        if Chunks[0] == '00':
            return RoundNumberDecoding('0' + ' '.join(Chunks[1:3])) + CompactDecoding0(Chunks[3])
        elif Chunks[0] == '01':
            return RoundNumberDecoding('0' + ' '.join(Chunks[1:3])) - CompactDecoding0(Chunks[3])
        elif Chunks[0] in ['000', '001']:
            Reference, SDelta = Decoding(' '.join(Chunks[1:])) # returns number + rest of string
    
```

```

        Delta, Rest = Decoding(SDelta)
        print(Reference, Delta)
        return Reference + Delta * (-1 if Chunks[0][-1] == '1' else 1)
    else: int('') # to raise ValueError
except (ValueError, IndexError): raise Exception('DifferenceDecoding error with %s' % SN)

def RoundReferenceCoding(N):
    " tries to find a round number in the vicinity "
    def rounding(N):
        SN = str(N)
        for order in range(1, len(SN)):
            yield (N // 10**order) * 10**order      # 12345 --> 12340, 12300, 12000, 10000
            yield (N // 10**order + 1) * 10**order  # 12345 --> 12350, 12400, 13000, 20000

    try:
        # print(N, [DifferenceCoding0(N, NRef) for NRef in rounding(N)])
        return min([DifferenceCoding0(N, NRef) for NRef in rounding(N)], key = Codelength)
    except ValueError: # possible for one-digit numbers
        return RoundNumberCoding(N)

#-----#
# Conditional coding #
#-----#

def ConditionalCoding(N, NRef):
    " N is coded by reference to NRef, and then the code for NRef is erased "
    DC = DifferenceCoding(N, NRef) # N is coding by difference to NRef
    CDC = DC.replace(Coding(NRef), '', 1).replace(' ', ' ') # erasing code of NRef
    CC = min([CDC, Coding(N)], key=len)
    if CC.split(' ')[0] in ['000', '001'] and len(CC) > 3: # real difference coding
        CC = '1' + CC[1:] # now prefix is 100 or 101
    return CC

def ConditionalDecoding(SN):
    " decodes N using last item put in memory "
    NRefC = Recall()
    Chunks = SN.split(' ')
    if Chunks[0] in ['100', '101'] and len(Chunks) > 1: # true conditional coding
        return Decoding('0% %s %s' % (Chunks[0][1:], NRefC, ' '.join(Chunks[1:])))[0]
    elif SN: return Decoding(SN)[0] # not a conditional code
    return Decoding(NRefC)[0] # empty conditional code

#####
# Using codes #
#####

# CODINGMETHODS = [CompactCoding, RoundNumberCoding, RoundReferenceCoding]
# CODINGMETHODS = [CompactCoding, RoundReferenceCoding]

Codelength = lambda Code: len(Code.replace(' ', '')) # separators are not counted !

def Coding(N):
    " returns the shortest available code "
    # print('Coding %d' % N)
    # return min([CM(N) for CM in CODINGMETHODS], key = len) # ordering by total length
    BestCode = min([CM(N) for CM in CODINGMETHODS], key = Codelength) # ordering by useful length
    return Memorize(BestCode) # put coded value in memory and return it

def Decoding(SN):
    " analyses a string and switches to specific codes "
    # print('Decoding < %s >' % SN)
    try:
        Chunks = SN.split(' ')
        # prefix decides which decoding is relevant
        if Chunks[0] == '1': return CompactDecoding(' '.join(Chunks[0:2])), ' '.join(Chunks[2:])
        if Chunks[0] == '0': return RoundNumberDecoding(' '.join(Chunks[0:3])), ' '.join(Chunks[3:])
        if Chunks[0] in ['00', '01']: return DifferenceDecoding(' '.join(Chunks[0:4])), ' '.join(Chunks[4:])
        if Chunks[0] in ['000', '001']: return DifferenceDecoding(SN), ''
        int('') # to raise exception
    except IndexError as Msg:
        print(Msg)
        raise Exception('Error with codechar %s' % SN)

def CodingTest(N, CodingMethod=Coding):
    " codes N and then decodes it "
    # NC = CodingMethod(N)
    # print(N, NC, DecodingMethod(NC))
    print('value: %d\tcoded: %s\tdecoded: %d' % (N, CodingMethod(N), Decoding(CodingMethod(N))[0]))

#####
# main #
#####

if __name__ == "__main__":
    print(__doc__)
    if len(sys.argv) > 1:
        try: Arg1 = int(sys.argv[1])
        except ValueError: print('Please input integer')
        if len(sys.argv) == 2: CodingTest(Arg1)
        else: # generation of codes for an interval of numbers
            # if Arg1 <= 0:
            #     print(usage())
            #     print('<min> should be >= 0')
            #     sys.exit(1)
            Max = int(sys.argv[2])
            # generating file
            F = open(OUTPUTFILE, 'w')
            F.write('N\tCode\tDecode\tTotal Length\tCode Length\tlog2(N)\n')
            for i in range(Arg1, Max+1):
                c = Coding(i)
                # F.write('%d\t%s\t%d\t%d\t%d\t%d\n' % (i, c, Decoding(c)[0], len(c), Codelength(c), ceil(log(i, 2))))
                F.write('%d\t%s\t%d\t%d\t%d\t%f\n' % (i, c, Decoding(c)[0], len(c), Codelength(c), log(i, 2) if i else 0))
            F.close()
            print(f'{OUTPUTFILE} created')
    else:
        print(usage())
        for i in range(12):
            CodingTest(i)

__author__ = 'Dessalles'
```